

# [S2] Data Warehousing and Online Analytical Processing

1. masih relevan, karena
2. kapan Hybrid (OLAP & OLTP)?
  - a. digabung klo butuh realtime (operational intellegence), dipisah ketika butuhkan
  - b.
3. 3 schema
  - a. star: klo butuh kecepatan query
  - b. snow: klo punya kapasitas DB yang minim
  - c. conselation: klo kompleks datanya

## 1. Relevansi *Data Warehouse* di Tengah Tren Sistem *Hybrid* (HTAP)

Meskipun sistem *hybrid* sangat canggih, konsep *Data Warehouse* (DW) **masih sangat relevan**. Berikut alasannya:

- **Kebutuhan Data Historis Jangka Panjang:** Sistem operasional berfokus pada data nilai saat ini (*current value data*). Namun, untuk pengambilan keputusan manajemen, seringkali dibutuhkan perspektif historis (misalnya data 5-10 tahun terakhir). Menyimpan seluruh data historis ini pada sistem operasional akan sangat membebani kinerja. DW memang dikonstruksi secara khusus (memiliki elemen rentang waktu/*time variant*) untuk menampung data historis secara masif.
- **Konsolidasi dari Berbagai Sumber (Heterogen):** Perusahaan jarang hanya memiliki satu aplikasi operasional. DW dibangun dengan mengintegrasikan banyak sumber data yang heterogen, seperti basis data relasional, *flat files*,

dan catatan transaksi *online*. Sistem *hybrid* biasanya hanya bisa menganalisis data yang ada di dalam *databasenya* sendiri.

- **Kualitas dan Integritas Data:** Data operasional seringkali memiliki perbedaan format atau bahkan kotor. Ketika data dipindahkan ke DW, ada tahap ekstraksi, pembersihan (*data cleaning*), transformasi, dan pemuatan. Proses ini memastikan konsistensi konvensi penamaan, struktur kode, dan ukuran atribut dari berbagai sumber, menghasilkan satu "versi kebenaran" yang siap dianalisis.

## 2. Kapan Sistem *Hybrid* Menjadi Tidak Relevan atau Kurang Cocok?

Sistem *hybrid* (menyatukan OLTP dan OLAP di platform yang sama) menjadi **tidak optimal** dalam beberapa kasus (penggunaan) berikut:

- **Analitik Skala Masif yang Memproses Jutaan Baris:** OLTP dirancang untuk memproses puluhan transaksi data secara repetitif, sedangkan OLAP dirancang untuk memindai (*scan*) jutaan baris data untuk kueri (*query*) kompleks yang bersifat *ad-hoc*. Jika *query* analitik yang sangat berat dijalankan di atas sistem operasional, hal ini akan memicu *resource contention* (perebutan sumber daya komputasi). Akibatnya, performa operasional harian (seperti pelanggan yang sedang bertransaksi) bisa melambat dan *lag*.
- **Transformasi Data yang Sangat Kompleks:** Desain *database* operasional berorientasi pada aplikasi (*application-oriented*), sedangkan desain OLAP berorientasi pada subjek tertentu (*subject-oriented*). Jika *business intelligence* membutuhkan perhitungan rumit (agregasi, *summarization*) sebelum data memiliki makna bisnis, melakukan ini secara langsung di atas sistem operasional secara terus-menerus sangat tidak efisien.
- **Kebutuhan Privasi dan Pemisahan Tugas:** Dalam beberapa aturan industri (seperti perbankan/medis), analis tidak boleh melihat raw data pengguna secara langsung (*Personally Identifiable Information*). DW menyediakan lingkungan terpisah yang fisik dan *nonvolatile* di mana data sudah disamarkan dan diubah strukturnya khusus untuk analis bisnis dan pekerja berpengetahuan (*knowledge worker*).

## 3. Penggunaan Optimal: Star, Snowflake, dan Fact Constellation

Di dalam pemodelan data warehouse konseptual, ketiga skema ini diciptakan untuk "medan tempur" yang berbeda:

- **Star Schema (Skema Bintang):** Skema ini memiliki sebuah tabel fakta (*fact table*) di tengah yang terhubung secara langsung ke sekumpulan tabel dimensi (*dimension tables*) di sekelilingnya.
  - **Optimal dalam kasus:** Prioritas utamanya adalah **kecepatan akses data (*query performance*)**. Karena bentuknya *denormalized*, mesin *database* hanya membutuhkan sedikit proses penghubungan antar tabel (*JOIN*). Ini adalah bentuk paling ideal dan cepat bagi *end-user* atau aplikasi pelaporan (seperti *dashboard*) yang ingin langsung membaca data agregat.
- **Snowflake Schema (Skema Kepingan Salju):** Ini adalah bentuk perbaikan dari *Star Schema*, di mana hierarki dari beberapa tabel dimensi dipecah atau dinormalisasi menjadi tabel-tabel dimensi yang lebih kecil.
  - **Optimal dalam kasus:** Terdapat kebutuhan untuk **mempermudah pembaruan data dimensi (*maintenance*)** dan ruang penyimpanan (meski saat ini biaya *storage* makin murah). Misalnya, sebuah atribut lokasi yang tadinya berada di satu tabel, dipecah menjadi *city* yang terhubung ke *location*. Optimal jika tabel dimensinya sangat besar dan rawan anomali jika diubah secara massal, asalkan performa mesin *database* Anda kuat menangani lebih banyak operasi *JOIN*.
- **Fact Constellation / Galaxy Schema (Konstelasi Fakta):** Skema komprehensif di mana beberapa tabel fakta berbagi tabel dimensi yang sama, yang jika dilihat akan berbentuk seperti koleksi bintang atau galaksi.
  - **Optimal dalam kasus:** Membangun **Data Warehouse skala perusahaan (*Enterprise Data Warehouse*)** dengan proses bisnis yang saling tumpang tindih. Misalnya, perusahaan ingin menganalisis "Penjualan" (Sales Fact Table) sekaligus "Pengiriman" (Shipping Fact Table). Kedua aktivitas bisnis ini dapat saling berbagai sudut pandang (*dimension tables*) yang sama, seperti tabel waktu (*time*), barang (*item*), dan lokasi (*location*). Sangat optimal untuk melihat korelasi operasional yang luas secara holistik.

data warehouse: gudang data

data warehouse vs database

secara definisi: data warehouse adalah data yang subjektif, terintegrasi, time-variant, dan nonvolatile yang digunakan untuk manajemen mengambil keputusan

database: transaction oriented



#### **DATA WAREHOUSE—INTEGRATED**

- Constructed by integrating multiple, heterogeneous data sources
  - relational databases, flat files, on-line transaction records
- Data cleaning and data integration techniques are applied.
  - Ensure consistency in naming conventions, encoding structures, attribute measures, etc. among different data sources
    - E.g., Hotel price: currency, tax, breakfast covered, etc.
  - When data is moved to the warehouse, it is converted.

data warehouse: integrated, datanya ga terpecah pecah, jadi misalnya ngomongin customer yauda itu semua yang di butuhkan terkait customer ya ada di tempat data customer. selain itu juga udah di adjust kayak naming conventions, encoding structures, attribute measures (misal semua seragam either cm, mm, or anything), etc

### DATA WAREHOUSE—TIME VARIANT

- The time horizon for the data warehouse is significantly longer than that of operational systems
  - Operational database: current value data
  - Data warehouse data: provide information from a historical perspective (e.g., past 5-10 years)
- Every key structure in the data warehouse
  - Contains an element of time, explicitly or implicitly
  - But the key of operational data may or may not contain “time element”

time-variant

ada id, timestamp

karena oneday klo data dibutuhkan berdasarkan waktunya, data yang ditarik hanya data yg relevan dengan timeframe tersebut. misal nyari data pajak 2005-2010, maka ketika di cari ya yg keluar hanya thn 2005-2010

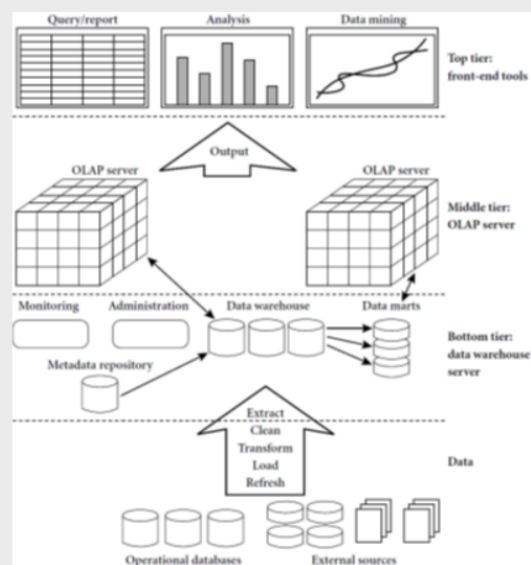
## DATA WAREHOUSE—NONVOLATILE

- A **physically separate store** of data transformed from the operational environment
- Operational **update of data does not occur** in the data warehouse environment
  - Does not require transaction processing, recovery, and concurrency control mechanisms
  - Requires only two operations in data accessing:
    - *initial loading of data* and *access of data*

misalnya uang fino udah nambah 20jt, tapi yang di data baru 10jt  
ya datanya masih 10jt, ini namanya snap shoot

## DATA WAREHOUSE: A MULTI-TIERED ARCHITECTURE

- Top Tier: Front-End Tools
- Middle Tier: OLAP Server
- Bottom Tier: Data Warehouse Server
- Data



multi-tiered architecture

ini namanya system architecture bukan software architecture

ini decouple system, agar mudah di maintenance

1 tier menghandle subsystem

kenapa jd subsystem, karena punya functional yg beda

proses butuh resource

resource ada 2, software dan hardware

knp system dipisah karena hardwarenya beda beda

- di tier bottom: heavy di read, dia butuh read yang cepat, di tier ini fokus nya ke hardware, disini ga butuh operation yang kompleks, ga butuh computational power yang besar
- klo middle: ga butuh storage yang besar (hdd/ssd), tapi butuhnya memory (RAM) karena butuh computation
- font-end: butuh memory besar untuk bisa cache, biar ga dikit dikit loading, butuh computation cuman ga yg gede gede, yang penting bisa komunikasi antar client bersamaan

## FROM TABLES AND SPREADSHEETS TO DATA CUBES

- A data warehouse is based on a **multidimensional data model** which views data in the form of a data cube
- A data cube, such as **sales**, allows data to be modeled and viewed in multiple dimensions
- Dimension tables, such as **item (item\_name, brand, type)**, or **time(day, week, month, quarter, year)**
- Fact table contains measures (such as **dollars\_sold**) and keys to each of the related dimension tables
- In data warehousing literature, an n-D base cube is called a **base cuboid**. The top most 0-D cuboid, which holds the highest-level of summarization, is called the **apex cuboid**. The lattice of cuboids forms a **data cube**.

butuh data struktur khusus untuk ...

krn database di simpan dalam Record

klo di data warehouse di simpan dalam varian tertentu (time variant atau space variant)

datacube: record yang 3 dimensi, jadi ada record 1 tahun, record 2 tahun, dst

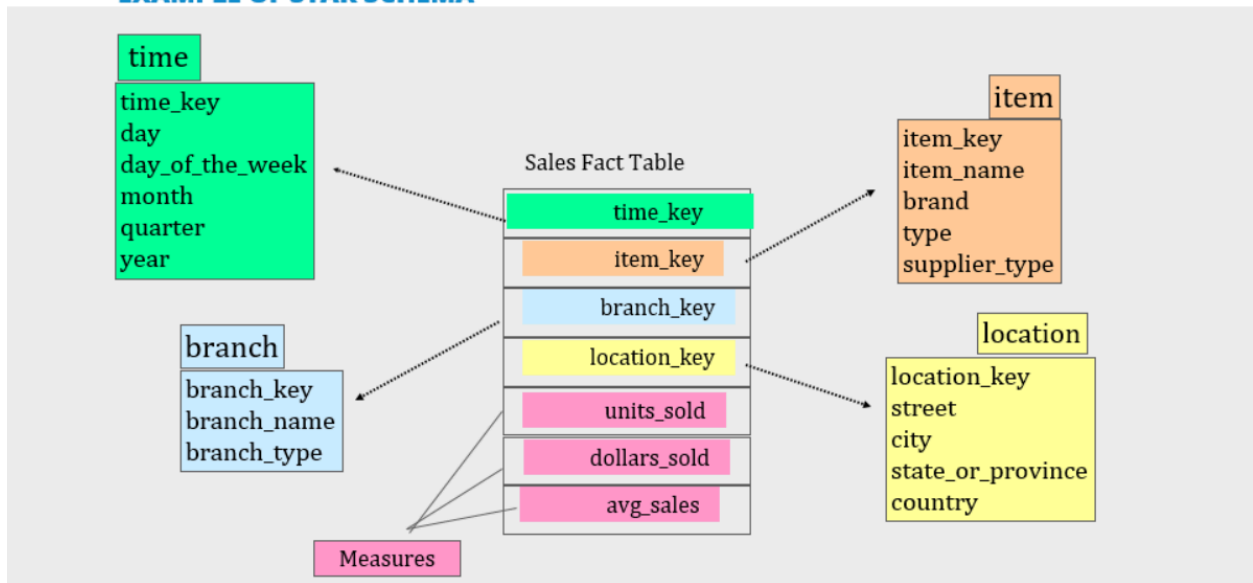
cube ini bukan hanya time, tapi bisa di

rollup: jadi seminggu jadi sebulan, bulan jadi tahun, dsb

drilldown: bulan jadi minggu, tahun jadi bulan, dsb

knp menarik? karena data warehous ini biasanya kan klo di database ada 1nf, 2 nf, justru di data ware house itu denormalisasi, karena lebih mudah ga banyak join join

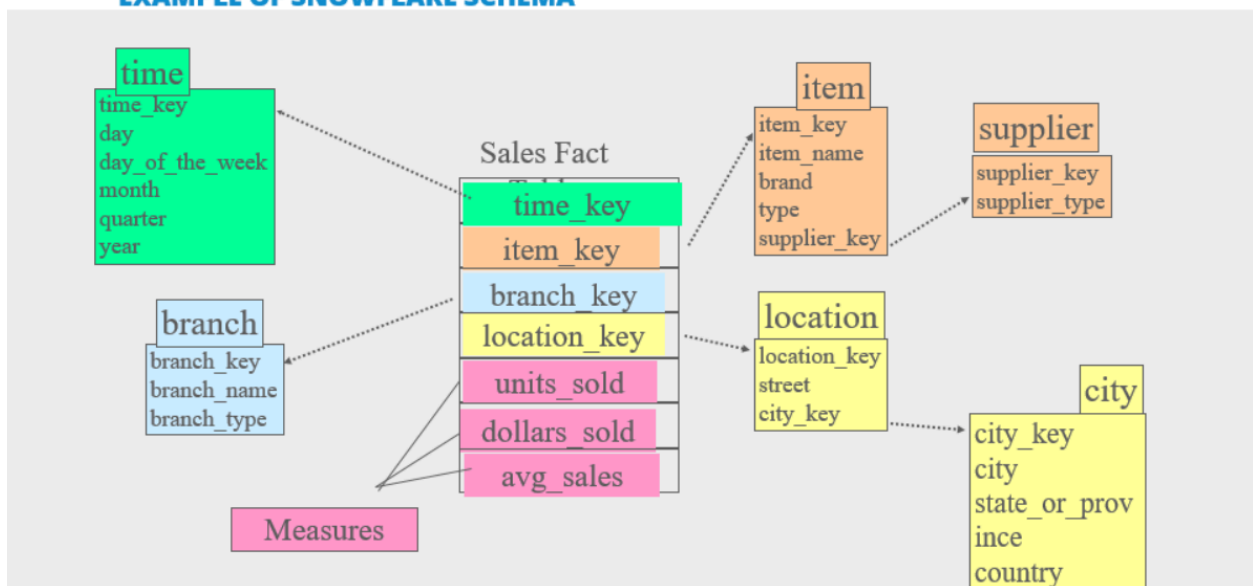
## EXAMPLE OF STAR SCHEMA



star schema, join semua tabel jadi 1 tabel, jadi di denormalisasi

kenapa? karena yang di cari adalah speed untuk retrieve data

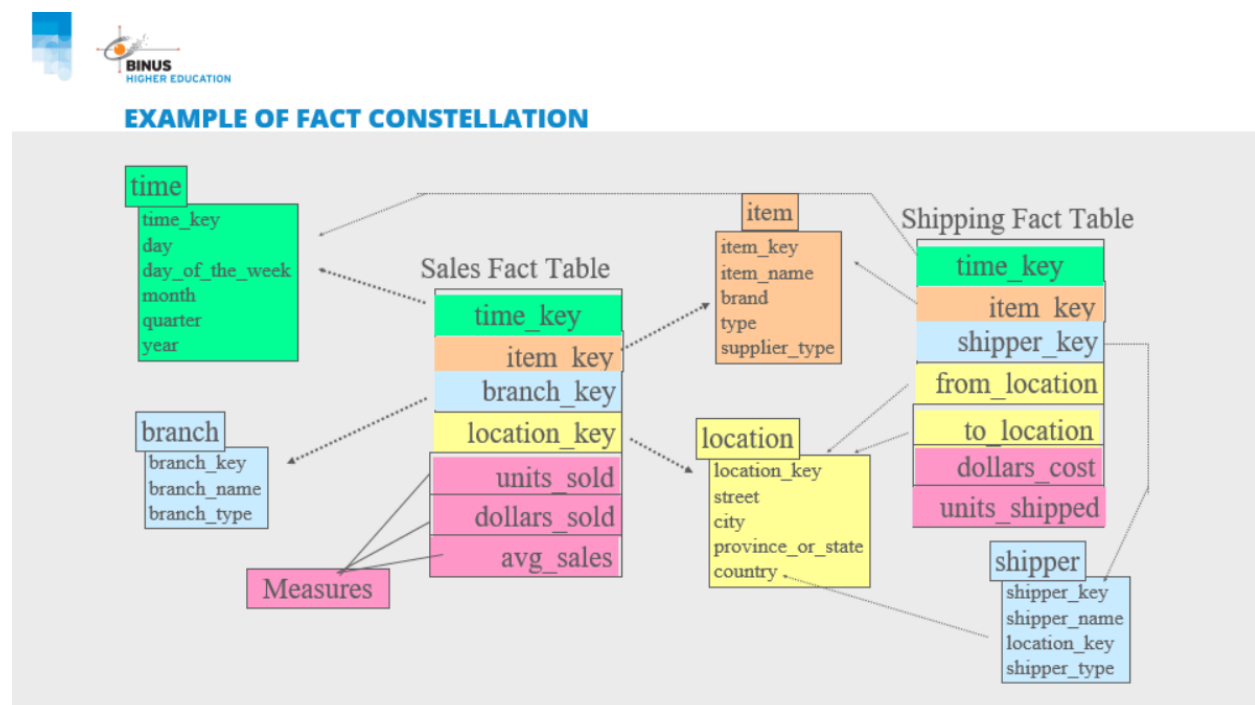
## EXAMPLE OF SNOWFLAKE SCHEMA



snowflake, bentuk lain dari star, karena yang di optimasi disini adalah struktur misal x1, x2, dan x3 jadi X

karena klo di denormalisasi di simpan semua ke 1 row, susah bedain itu dari info x1 atau x2

supaya bisa nge preserve hal tersebut, pakai snowflake, bisa preserve strukture dari yang kita miliki



fact constellation

ada 2 system yang terpisah, disini yang di jaga adalah integrasi

misal ada bintang utara (informasi cells) dan bintang selatan (informasi gudang)

ketika 2 data ini mau di join, kita pakai fact constellation,

yang di optimasi disini adalah menjaga speed dan menjaga identitas business nya (entitasnya)

## OLTP VS. OLAP

|                           | OLTP                                                         | OLAP                                                                    |
|---------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>users</b>              | clerk, IT professional                                       | knowledge worker                                                        |
| <b>function</b>           | day to day operations                                        | decision support                                                        |
| <b>DB design</b>          | application-oriented                                         | subject-oriented                                                        |
| <b>data</b>               | current, up-to-date<br>detailed, flat relational<br>isolated | historical,<br>summarized, multidimensional<br>integrated, consolidated |
| <b>usage</b>              | repetitive                                                   | ad-hoc                                                                  |
| <b>access</b>             | read/write<br>index/hash on prim. key                        | lots of scans                                                           |
| <b>unit of work</b>       | short, simple transaction                                    | complex query                                                           |
| <b># records accessed</b> | tens                                                         | millions                                                                |
| <b>#users</b>             | thousands                                                    | hundreds                                                                |
| <b>DB size</b>            | 100MB-GB                                                     | 100GB-TB                                                                |
| <b>metric</b>             | transaction throughput                                       | query throughput, response                                              |

OLTP (on-line transaction processing) isinya data sehari hari, misal ketika ada yg masukin keranjang, checkout, dsb

OLAP (on-line analytical processing)

kapan hybrid (OLTP & OLAP)? ketika ingin decision support yang secara realtime, "OLTP nya itu sebagai fondasi untuk kontruksi system baru yang sejatinya itu OLAP itu sendiri"

kapan harus dipisah, ketika tidak ingin membebankan system

entitas: melekat ke suatu yang tidak terlepas, misalnya manusia itu entitas

subjek: ketika punya predikat, manusia bisa jadi subjek ketika punya predikat

Project

dashboard interaktif

latensi maximal 100ms